

Test-Time Graph Search for Goal-Conditioned Reinforcement Learning

Anonymous Authors¹

Abstract

Offline goal-conditioned reinforcement learning (GCRL) often struggles with long-horizon tasks, where errors in value estimation accumulate and produce unreliable policies. It is typically assumed that effective long-term planning is infeasible without specialized training. In contrast, our work demonstrates that existing GCRL policies can complete long-horizon tasks when combined with a lightweight, training-free planning wrapper. We find that standard goal-conditioned value functions encode locally consistent geometric structure sufficient for planning. Our approach, Test-Time Graph Search (TTGS), constructs a graph over the offline dataset and employs an adaptive subgoal selection strategy. To address unreliable value estimates during shortest-path search, we propose a novel mechanism that softly penalizes long-distance transitions. Our method incurs negligible computational overhead and requires no additional supervision or parameter updates. On the OGBench benchmark, TTGS unlocks latent capabilities in diverse base learners, boosting success rates on challenging locomotion tasks from near-zero to over 90%, often matching or outperforming methods that require complex auxiliary training.

1. Introduction

Goal-conditioned reinforcement learning trains agents to reach user-specified goals and offers a framework for extracting diverse behaviors from large datasets. The goal specification decouples behavior from specific task rewards, which enables reuse across many objectives and facilitates broad data curation without fragile reward engineering (Schaul et al., 2015; Andrychowicz et al., 2017). In the offline regime, GCRL can leverage precollected data in domains

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

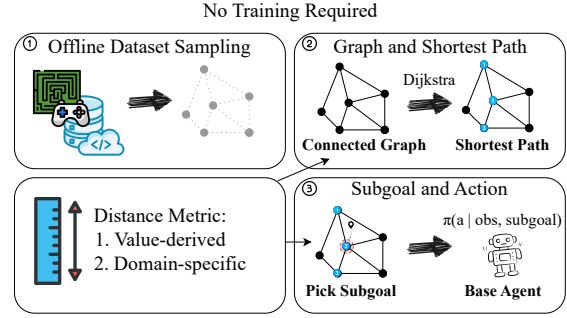


Figure 1. **Overview of TTGS.** From an offline dataset, we sample observations to form graph vertices. We assign edge weights using a distance signal, either derived from a pretrained goal-conditioned value function or from domain-specific knowledge. A shortest-path search with Dijkstra’s algorithm yields a sequence of subgoals that guides a frozen policy at test time.

where online interaction is expensive, time-consuming, or unsafe, such as robotics, healthcare, and autonomous driving (Fu et al., 2020; Abdellatif et al., 2021; Kumar et al., 2020).

Despite these advantages, long-horizon decision-making remains challenging. Recent evaluations report that methods performing well in moderate-horizon environments struggle in larger spaces, such as complex mazes (Park et al., 2025; Sobal et al., 2025). To bridge this gap, prior solutions have introduced significant complexity: deploying computationally intensive generative planners (Ajay et al., 2023; Luo et al., 2025), training distributional value function ensembles (Eysenbach et al., 2019), utilizing specialized geometric representations (Baek et al., 2025), or relying on additional data beyond the offline dataset (Emmons et al., 2020). These methods typically operate on the implicit assumption that standard goal-conditioned value functions are inherently too unreliable or noisy to support long-horizon planning directly.

In contrast, we demonstrate that this additional complexity is often unnecessary. We show that standard goal-conditioned value functions, while prone to global errors, already encode consistent local geometric structure sufficient for long-horizon planning. By processing this signal with a soft-penalty mechanism, we can guide search over dataset states to yield a path where each segment remains within the capa-

bilities of the pretrained frozen policy, without the need for specialized training. We further show that this framework generalizes to domain-specific heuristics, such as Euclidean proximity.

We introduce Test-Time Graph Search (TTGS), a lightweight, training-free planning wrapper that augments frozen GCRL policies. This design targets the common setting where a practitioner has already invested substantial effort in training and tuning a base goal-conditioned agent, but finds that it fails to reliably solve longer-horizon tasks at deployment. TTGS offers a simple way to improve long-horizon performance by reusing the existing policy and its learned value function without changing the training pipeline.

Unlike prior graph-based reinforcement learning methods that focus on training specialized distance functions or pruning data via online interaction (Emmons et al., 2020), TTGS operates entirely at test time. It constructs a graph over the pretraining dataset, assigns edge costs using an existing distance function, and performs a fast shortest-path search to generate a sequence of intermediate subgoals. The adaptive subgoal selection feeds reachable subgoals ahead of the agent to the frozen policy. We also show that this framework is flexible: while our primary focus is unlocking the latent geometry of value functions, TTGS seamlessly accepts domain-specific heuristics (e.g., Euclidean distance) when available. In both cases, planning over the induced geometry significantly improves trajectory stitching and long-horizon performance without modifying the underlying policy, requiring additional training, or online interaction.

TTGS integrates with existing goal-conditioned learners purely as a test-time procedure. It requires only a distance signal and an offline dataset, adds minimal computational overhead, and preserves the behavior learned by the base policy. The framework complements both hierarchical and non-hierarchical offline GCRL methods by supplying explicit long-horizon search at test time. Since many GCRL algorithms already learn goal-conditioned value functions, the value-derived distance is a natural default that keeps the method simple to adopt.

We make the following contributions:

- **Re-evaluation of Value Functions for Planning:** We demonstrate that standard offline GCRL value functions, previously considered too unreliable for search, can support robust long-horizon planning when processed with a novel soft-penalty mechanism.
- **Test-Time Graph Search (TTGS):** We propose a lightweight, training-free framework that augments frozen agents with graph search. Unlike prior graph-based approaches, TTGS is designed to work with existing policies, requiring no additional supervision or online interaction.

- **Performance Improvement With Minimal Overhead:** We show empirically that TTGS improves the performance of state-of-the-art offline GCRL agents (HIQL, QRL, GCIQL, SAW, and OTA) with negligible computational overhead (< 1 second planning time; see Appendix F). On OGBench long-horizon tasks, TTGS achieves performance competitive with more complex methods that require training or generative planning (Park et al., 2025; Luo et al., 2025; Baek et al., 2025).

2. Goal-Conditioned Reinforcement Learning Preliminaries

Following Park et al. (2025), we consider the offline goal-conditioned reinforcement learning (GCRL) problem, defined over a controlled Markov process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mu, p)$, i.e., a Markov Decision Process (MDP) without rewards, together with an unlabeled dataset $\mathcal{D}$ of trajectories. Here, $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $\mu \in \Delta(\mathcal{S})$ the initial state distribution, and $p(s' | s, a) : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ the transition dynamics. We denote by $\Delta(\mathcal{X})$ the space of probability distributions over a set $\mathcal{X}$.

The offline dataset $\mathcal{D} = \{\tau^{(n)}\}_{n=1}^N$ consists of trajectories $\tau^{(n)} = (s_0^{(n)}, a_0^{(n)}, s_1^{(n)}, \dots, s_{T_n}^{(n)})$ collected by some unknown behavior policy.

The goal in GCRL is to learn a goal-conditioned policy $\pi(a | s, g) : \mathcal{S} \times \mathcal{S} \rightarrow \Delta(\mathcal{A})$ that can efficiently reach any goal $g \in \mathcal{S}$ from any starting state $s \in \mathcal{S}$. Formally, the optimal policy maximizes the expected discounted return:

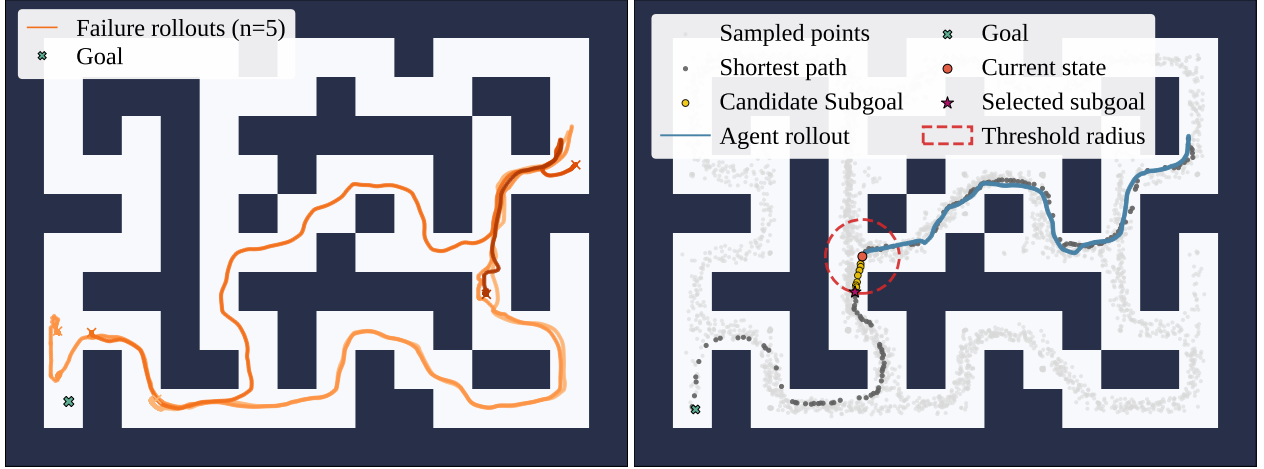
$$\max_{\pi} \mathbb{E}_{\tau \sim p(s|g)} \left[\sum_{t=0}^T \gamma^t \mathbf{1}\{\|s - g\| < \epsilon\} \right],$$

where T is the horizon, $\gamma \in (0, 1)$ the discount factor, $p(\tau | g)$ the trajectory distribution induced by π , and reward is equal to $\mathbf{1}\{\|s - g\| < \epsilon\}$. A common alternative formulation uses $\mathbf{1}\{\|s - g\| < \epsilon\} - 1$ reward.

Many GCRL algorithms, including HIQL (Park et al., 2023), GCIQL (Kostrikov et al., 2022), and QRL (Wang et al., 2023), learn a value function $V(s, g) : \mathcal{S} \times \mathcal{S} \rightarrow \mathbb{R}$ representing the expected discounted reward when navigating from state s to goal g .

3. Method

We visualize the core challenge that GCRL agents face in Figure 2. In long, complex tasks like maze navigation, agents that attempt to reach far-away goals can get stuck or run off course. However, for shorter horizons, they tend to be reliable. From this observation, we derive the simple key idea behind TTGS: rather than asking a policy to solve a long-horizon task in one shot, we decompose it into short, reliable hops. We do so by equipping test-time planning



(a) Rollouts from HIQL fail to reach a distant goal.

(b) TTGS selects reachable subgoals on a guiding path.

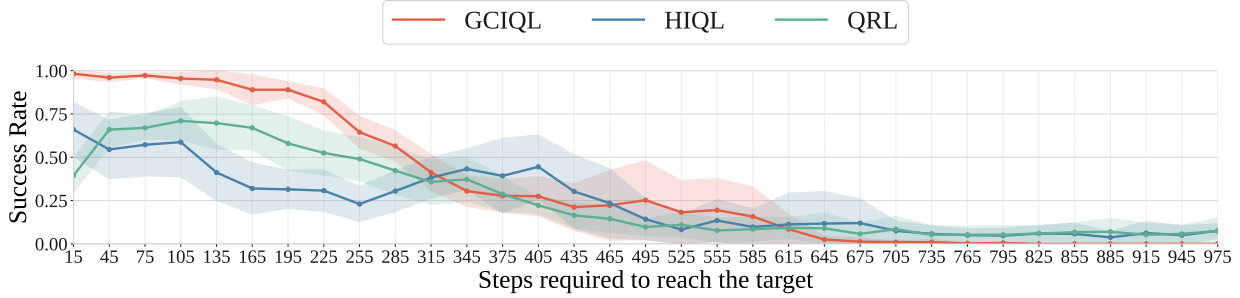

 (c) Success rate of reaching goals sampled from the expert trajectory and located n steps away from the agent within $1.5n$ step budget in antmaze-giant-stitch-v0 environment.

Figure 2. Motivation for TTGS: (a) HIQL policy fails to reach a distant goal on antmaze-giant-stitch-v0, with multiple attempts failing to exit the starting area and two attempts running out of time due to inefficient path. (b) TTGS finds a guiding path using dataset observations. On each step it selects a subgoal which is within a predefined radius from the agent. We mark all data points on the guiding path in gray, and the actual path traversed by the agent in blue. (c) Different agents’ policy performance decreases as steps required to reach the goal increase. By providing a policy with close subgoals, TTGS improves reliability and efficiency of reaching the goal.

with a distance signal over states, selecting a compact set of states from an offline dataset, and connecting them into a graph whose edges reflect predicted step costs. At test time, we plan in this graph by computing a shortest path between the start and the goal, then feed the policy a small number of intermediate subgoals along this path. As summarized in Figure 2b, the agent queries all points along the shortest path that are within a certain threshold from the current state, and picks the closest to the goal.

While the framework supports any compatible distance, our primary focus is the general and widely applicable case where the distance is derived from a learned goal-conditioned value function. The following sections describe how distances are calibrated (Section 3.1), how the graph is constructed (Section 3.2), and how subgoal-based planning is executed at test time (Section 3.3).

3.1. Distance Prediction

The algorithm begins by obtaining a distance predictor from an available (approximate) metric on the state space. In general, this can be any metric, here we focus on deriving approximate metrics from value function estimates. Note that in general these predictors do not fulfill the formal criteria for metrics, e.g. they might not obey the triangle inequality or be symmetric. However, we find them to be reliable for local reachability (i.e., within a short horizon), even if global long-horizon estimates are noisy, as supported by the effectiveness of TTGS in chaining these local estimates.

Value-derived distance (default). To enable policy-agnostic planning, we map a goal-conditioned value signal $V(s, g)$ to an expected step distance

$$\hat{d}(s, g) = f(V(s, g)),$$

where $f : \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$ converts values into an estimate of the environment steps required to reach g from s . For the following, common reward conventions, f admits a closed form:

- *Sparse terminal reward.* Reward 1 only at the goal and 0 otherwise. Then $V^*(s, g) = \gamma^d$, where d is the shortest-step distance, giving

$$\hat{d}(s, g) = \log_{\gamma} V^*(s, g).$$

- *Per-step penalty.* Reward -1 until reaching the goal and 0 at the goal. Then

$$V^*(s, g) = -\sum_{t=0}^{d-1} \gamma^t = -\frac{1 - \gamma^d}{1 - \gamma}$$

$$\Rightarrow \hat{d}(s, g) = \log_{\gamma}(1 + (1 - \gamma) V^*(s, g)).$$

HIQL, SAW and GCIQL use this convention in our experiments.

More generally, any value signal that is monotone with respect to reachability suffices. We clip $\hat{d}$ to avoid negative or infinite outputs. Exact mappings for each base agent are provided in [Appendix B](#).

Domain-specific distances. When a task supplies a meaningful geometric or kinematic metric, we can use it directly as $\hat{d}$. This requires no changes to the pipeline below; only the source of edge costs differs. Our experiments include such a domain-specific metric to illustrate the flexibility of the framework.

3.2. Graph Construction

Using the distance predictor, the agent now proceeds to build the underlying graph of distances from the pretraining dataset. This graph is goal-agnostic, which means it can be reused for different goals an agent might want to achieve in the environment.

Vertices selection Computing all pairwise distances among N states in $\mathcal{D}$ has $O(N^2)$ complexity and may require repeated neural network evaluations, which is prohibitive at scale. We instead sample a compact subset $\mathcal{V} = \{v_i\}_{i=1}^M \subset \mathcal{D}$ and build the graph on $\mathcal{V}$. Uniform random sampling worked well in our experiments. We also tested clustering inspired by [Baek et al. \(2025\)](#) (see [Appendix D](#) for details), but it did not produce a statistically significant gain over random sampling, so we adopt random sampling for simplicity.

Weights derivation We build a directed, weighted graph $G = (\mathcal{V}, E, \tilde{w})$. Using the distance predictor $\hat{d}$, we compute a matrix $D \in \mathbb{R}^{M \times M}$ with entries $D_{ij} = \hat{d}(v_i, v_j)$

Algorithm 1 TTGS: Subgoal Selection and Execution

Require: Graph $G = (\mathcal{V}, E, \tilde{w})$ where $\mathcal{V} = \{v_i\}_{i=1}^M$ are selected states; distance $\hat{d}$; frozen policy π ; start state s_0 ; goal g ; step budget T

- 1: {Precompute guide path once per episode}
- 2: $v_s \leftarrow \arg \min_{v_i} \hat{d}(s_0, v_i)$, $v_g \leftarrow \arg \min_{v_i} \hat{d}(v_i, g)$
- 3: $(p_0, \dots, p_L) \leftarrow \text{DIJKSTRA}(G, v_s, v_g)$
- 4: $k_{\text{prev}} \leftarrow 0$; $s_{\text{cur}} \leftarrow s_0$
- 5: **while** g not reached **do**
- 6: $\delta_{\ell} \leftarrow \hat{d}(s_{\text{cur}}, p_{\ell})$ for $\ell = 0, \dots, L$; $\delta_g \leftarrow \hat{d}(s_{\text{cur}}, g)$
- 7: $k \leftarrow \arg \min_{\ell} \delta_{\ell}$; $k \leftarrow \max(k, k_{\text{prev}})$ {index of the closest waypoint ahead}
- 8: **if** $\delta_g \leq T$ **then**
- 9: $\tilde{g} \leftarrow g$ {goal is within reach}
- 10: **else**
- 11: $\mathcal{C} \leftarrow \{\ell > k : \delta_{\ell} \leq T\}$ {reachable subgoal candidates}
- 12: **if** $\mathcal{C} \neq \emptyset$ **then**
- 13: $\tilde{g} \leftarrow p_{\max \mathcal{C}}$ {take farthest reachable}
- 14: **else**
- 15: $\tilde{g} \leftarrow p_{\min\{k+1, L\}}$ {take next subgoal}
- 16: **end if**
- 17: **end if**
- 18: Sample $a \sim \pi(\cdot | s_{\text{cur}}, \tilde{g})$ and execute.
- 19: $s_{\text{cur}} \leftarrow$ next observed state; $k_{\text{prev}} \leftarrow k$
- 20: **end while**

using batched evaluations for efficiency. Because distance predictors can overestimate connectivity across gaps (creating “wormholes”), minimizing raw distance often yields infeasible paths. Conversely, enforcing a hard distance limit risks fragmenting the graph (see [Appendix E](#)). To balance these issues, we assign edge weights $\tilde{w}_{ij}$ using a superlinear penalty $p(\cdot)$ for transitions exceeding a trust region τ :

$$\tilde{w}_{ij} = \begin{cases} D_{ij}, & i \neq j, D_{ij} < \tau, \\ p(D_{ij}), & i \neq j, D_{ij} \geq \tau, \\ +\infty, & i = j. \end{cases}$$

Self-loops are removed to avoid unnecessary computations. In experiments we use $p(x) = x \cdot 1000^{x/\tau}$. This penalty preserves local metric structure below τ while steering the planner toward chains of short, trustworthy hops, utilizing longer edges only when no safer path exists.

3.3. Subgoal Selection and Action Sampling

To find the path to a given goal, the agent proceeds in two steps based on the pre-computed graph.

Shortest-path precomputation. At test time, given a start state s_0 and a goal g , we compute a guide path on G using



Figure 3. Goal-reaching success rates for QRL, GCIQL, HIQL, SAW and OTA with and without TTGS. Distances are predicted from each base agent’s learned value function. TTGS improves performance on the majority of locomotion tasks that require trajectory stitching. Full results are presented in Table 3.

Dijkstra’s algorithm. We first locate the nearest vertices under $\hat{d}$,

$$v_s = \arg \min_{v_i} \hat{d}(s_0, v_i), \quad v_g = \arg \min_{v_i} \hat{d}(v_i, g),$$

then obtain $(p_0, \dots, p_L) = \text{DIJKSTRA}(G, v_s, v_g)$. The guide path is computed once per episode and reused throughout execution. Using a fast GPU-based implementation of Dijkstra’s algorithm keeps the overhead minimal.

Adaptive subgoal selection. During execution, the agent selects subgoals from $(p_0, \dots, p_L)$ using a step budget T . If the current state s_{cur} is within T steps of g according to $\hat{d}$, the subgoal is set to g . Otherwise, the agent identifies the closest waypoint to the current state and updates a progress index k to ensure monotonicity, prohibiting the reference point from moving backward (i.e., $k \leftarrow \max(k, k_{\text{prev}})$). It then chooses the farthest waypoint p_j with index $j \geq k$ that is within budget, $\hat{d}(s_{\text{cur}}, p_j) < T$. If no such waypoint exists, the next waypoint along the path is chosen to ensure forward progress. The frozen goal-conditioned policy π is then invoked with current state and selected subgoal. The pseudocode is provided in Algorithm 1.

4. Experiments

We evaluate TTGS on OGBench (Park et al., 2025) using three strong offline GCRL baselines provided by the benchmark: QRL, GCIQL, and HIQL (Wang et al., 2023; Park et al., 2023). Additionally, we include SAW (Zhou & Kao, 2025) and OTA (Ahn et al., 2025) to broaden the evaluation. For each dataset, we compare the success rate of the frozen base policy with and without TTGS across five tasks. Every task is evaluated with 50 rollouts, and we report the mean and standard deviation over eight random seeds. In tables, methods within 95% of the best mean are bolded, and cases where TTGS exceeds its corresponding base learner are underlined. All base agents are trained with the public OGBench code using default hyperparameters, and TTGS is applied post hoc without any changes to training.

Datasets OGBench provides offline datasets for GCRL, including locomotion domains that test long-horizon

and hierarchical reasoning (Park et al., 2025). We use pointmaze, antmaze, and humanoidmaze with medium, large, and giant mazes and three data-collection regimes: navigate uses a noisy expert reaching random goals, stitch consists of short trajectories that require trajectory stitching, and explore contains random exploratory rollouts (available only for antmaze). For antmaze and humanoidmaze, visual variants with pixel-based observations are also available.

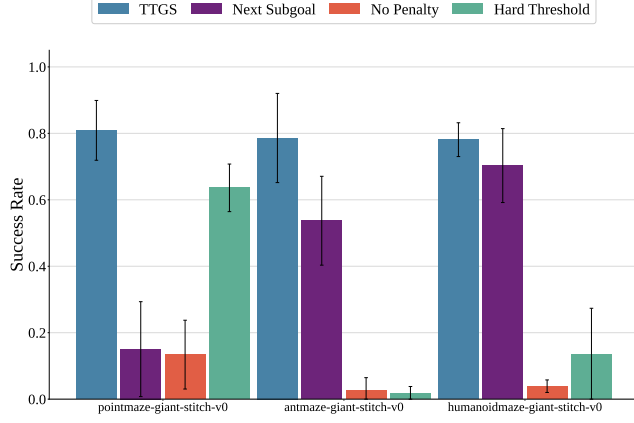
4.1. Main Results

Figure 3 summarizes results on the largest stitching tasks, which are challenging for existing agents. TTGS uses distances derived from the base agent’s value function, so it relies only on information available at test time. TTGS generally improves or maintains success across evaluated environments, with the largest gains on giant layouts where one-shot execution is difficult. On pointmaze-giant-stitch-v0, TTGS raises HIQL from 0.0% to 80.9% and GCIQL from 0.0% to 98.0%. On humanoidmaze-giant-stitch-v0, HIQL is improved from 4.4% to 78.1%. These cases highlight how sequences of short, reliable hops help frozen policies traverse large mazes that require stitching behaviors from disparate parts of the dataset. Notably, TTGS boosts even HIQL, SAW, and OTA which already include a learned high-level controller, suggesting that explicit planning can provide more reliable subgoals than a purely learned subgoal policy in long-horizon settings. We further observe improvements on several pixel-based tasks, indicating that value-derived distances can offer a strong planning signal even when domain-specific metrics are difficult to define. While Figure 3 highlights the hardest stitch tasks, we provide comprehensive results for all evaluated environments, including medium, large, and giant mazes across stitch, navigate, and explore settings, in Table 3 of Appendix A. We further analyze the applicability of TTGS to manipulation tasks in Appendix C, finding that the method behaves robustly even when value landscapes are sparse.

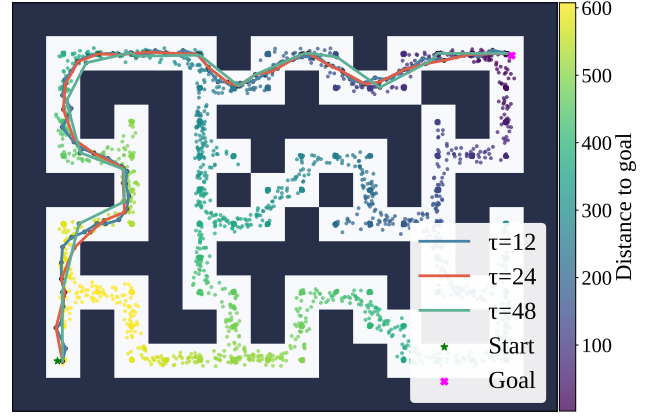
To test the generality of TTGS beyond value-based distances, we also evaluate a simple domain-specific distance based

Table 1. HIQL+TTGS on state-based datasets using value-derived and position-based (L_2) distances. The simple Euclidean distance between body positions, normalized by the average dataset step length, is often sufficient to yield strong gains.

Dataset	HIQL	HIQL+TTGS-value	HIQL+TTGS- L_2
pointmaze-giant-navigate-v0	43.0 $\pm$ 10.5	70.9 $\pm$ 12.2	70.0 $\pm$ 9.3
pointmaze-giant-stitch-v0	0.0 $\pm$ 0.0	80.9 $\pm$ 9.0	83.0 $\pm$ 7.8
antmaze-giant-navigate-v0	65.0 $\pm$ 4.1	65.8 $\pm$ 4.0	60.8 $\pm$ 4.3
antmaze-giant-stitch-v0	1.4 $\pm$ 1.1	78.6 $\pm$ 13.4	60.2 $\pm$ 12.0
antmaze-large-explore-v0	2.4 $\pm$ 4.4	26.6 $\pm$ 34.0	18.8 $\pm$ 23.8
humanoidmaze-giant-navigate-v0	16.0 $\pm$ 8.6	85.3 $\pm$ 6.1	74.2 $\pm$ 17.5
humanoidmaze-giant-stitch-v0	4.4 $\pm$ 1.3	78.1 $\pm$ 5.1	72.6 $\pm$ 5.7



(a) Ablations of HIQL+TTGS-value.



(b) Effect of the threshold τ and predicted distances.

Figure 4. **Ablations and hyperparameters.** (a) Comparison of full TTGS using HIQL as base learner and value-derived distances with three ablations: *Next Subgoal* replaces our subgoal selection procedure with always picking the immediate next waypoint, *No-Penalty* uses raw predicted distances, and *Hard-Threshold* removes edges with cost $> \tau$. TTGS outperforms both ablations across datasets. (b) Effect of penalty threshold τ on the guide path and a value-derived distance field. Colors denote predicted distances from each dataset observation to the goal in top-right corner. Smaller τ yields denser subgoals and less direct paths. Larger τ permits longer hops that can require navigating around obstacles, which is harder for the frozen policy.

on body position. This metric uses the Euclidean distance between agent body positions normalized by the average step length in the dataset. Although it ignores walls, joint configuration and contact dynamics, this signal is sufficient to unlock substantial gains when combined with TTGS. Table 1 reports that HIQL+TTGS with this metric improves performance across most state-based environments, which supports the claim that TTGS can exploit a variety of compatible distances when a value function is unavailable.

4.2. Ablation Study

We study two design choices: adaptive subgoal selection and the edge-length penalty introduced during graph construction. We also examine sensitivity to the distance penalty threshold τ and the subgoal selection threshold T .

Subgoal selection. Our default procedure selects the farthest waypoint on the precomputed route whose predicted distance from the current state is below a threshold T , falling back to the next waypoint after the closest node when no such waypoint exists. The alternative always com-

mits to the immediate next waypoint. Figure 4a shows that this simpler strategy degrades performance, especially on pointmaze-giant, while the adaptive rule yields robust improvements. We hypothesize that aiming at the farthest reachable waypoint reduces unnecessary micromanagement of local motion and promotes efficient progress toward reachable, yet sufficiently distant, subgoals.

Distance penalty. We compare the dynamic soft penalty from Section 3.2 to a *No-Penalty* variant (raw distances) and a *Hard-Threshold* variant (removing edges $> \tau$). Figure 4a shows that on stitching tasks, the soft penalty significantly outperforms the no-penalty baseline. Furthermore, as detailed in Appendix E, a hard threshold fails on these tasks because noisy value estimates frequently cause graph disconnections. The soft penalty maintains connectivity by assigning high costs to optimistic edges rather than removing them, allowing the planner to recover from local value function errors.

Table 2. TTGS consistently boosts HIQL on long-horizon stitch tasks. Success rate for HIQL with and without the TTGS on giant-stitch tasks as we vary τ and T .

Dataset	τ	T	HIQL	HIQL+TTGS
pointmaze-giant-stitch-v0	12	24	0.0 ± 0.0	78.4 ± 10.3
	24	24		57.2 ± 12.7
	24	48		80.9 ± 9.0
	24	96		62.6 ± 16.7
	48	96		52.0 ± 20.0
antmaze-giant-stitch-v0	12	24	1.4 ± 1.1	78.6 ± 13.4
	24	24		40.9 ± 14.6
	24	48		19.0 ± 9.2
	24	96		2.2 ± 1.6
	48	96		1.1 ± 1.5
humanoidmaze-giant-stitch-v0	12	24	4.4 ± 1.3	52.2 ± 6.1
	24	24		65.5 ± 12.0
	24	48		78.1 ± 5.1
	24	96		79.8 ± 7.7
	48	96		76.8 ± 9.7

Hyperparameters. We sweep the edge threshold τ and the subgoal threshold T . Table 2 reports HIQL results on the three largest state-based mazes. TTGS outperforms the base policy or preserves performance across all tested settings, with a moderate penalty threshold and a subgoal selection threshold near $T = 2\tau$ generally working best. Small τ values bias the planner toward long chains of short hops, which can lengthen routes and reduce straight-line progress, whereas large τ values admit long edges that the policy may fail to traverse. Crucially, this trade-off is interpretable without online interaction: τ can be selected by analyzing the suggested shortest paths to identify infeasible shortcuts or excessive path fragmentation. In an online setting, tuning is further simplified by the method’s training-free nature, which allows for fast iteration.

Replanning. We also investigate the effect of online replanning in Appendix G. We find that the initial guide path is generally robust, and frequent replanning typically offers limited benefit.

Finally, Figure 4b visualizes the value-derived distance field together with guide paths produced by TTGS. The distances obtained from the value function reflect the environment’s structure, with obstacles forming high-cost barriers and corridors forming low-cost channels. As the edge threshold τ decreases, the planner trusts only short edges and yields denser routes with more subgoals; as τ increases, longer edges are permitted, producing sparser but potentially less reliable paths.

5. Related Work

Offline goal-conditioned reinforcement learning (GCRL) aims to learn multi-task policies from fixed datasets. A key challenge is to infer reliable goal-conditioned values from

imperfect data. Prior approaches stabilize value estimation with expectile regression (Kostrikov et al., 2022; Park et al., 2023), treat values as quasi-metric distances (Wang et al., 2023), couple representation learning with one-step improvement via contrastive objectives (Eysenbach et al., 2022; Zheng et al., 2024), incorporate temporal abstraction into value learning (Ahn et al., 2025), or impose physics-informed regularization derived from the Eikonal equation (Giammarino et al., 2025). These methods perform well on moderate horizons but often fail on tasks that require trajectory stitching. TTGS is designed to complement them by supplying test-time planning to improve their long-horizon performance.

Graph-based methods have also been explored in GCRL. Some use graph search to prune suboptimal actions from datasets (Yin & Abbeel, 2024), while others construct graphs from episodes in a learned metric space and apply reinforcement learning over the graph (Zhu et al., 2023). Several works focus on learning more accurate distance estimators, including temporal distance representations (Baek et al., 2025), ensembles of distributional value functions (Eysenbach et al., 2019), Siamese proximity predictors (Savinov et al., 2018), or latent landmarks (Zhang et al., 2021). Some approaches estimate distances via value functions but require online refinement to prune infeasible edges (Emmons et al., 2020). Unlike prior graph search approaches, TTGS requires no training, it treats pretrained policies and distance estimators as black boxes and operates entirely offline. We show that instead of using specialized training or online pruning, applying a simple soft threshold and subgoal selection on top of standard pretrained agents is sufficient to gain the benefits of graph search.

Beyond graph search, several planning paradigms have been proposed. Diffusion and generative models compose or aug-

ment trajectories for long-horizon planning (Janner et al., 2022; Ajay et al., 2023; Luo et al., 2025; Zhang et al., 2025; Lee & Choi, 2025), though they typically demand extra training and computation. Model-based planning uses learned dynamics for model-predictive control (Williams et al., 2017; Chua et al., 2018; Hafner et al., 2019; Hansen et al., 2022) or sequence-model planning with trajectory transformers (Janner et al., 2021). TTGS differs by providing fast (< 1 s planning time), training-free retrieval of the dataset states, which can complement generative and model-based methods by providing reliable subgoals.

6. Limitations

TTGS introduces some computational overhead, although it remains modest. Graph construction is performed once per dataset and requires roughly 35 to 100 seconds on a single Nvidia L40s GPU with 8 virtual CPU cores, depending on the domain. At test time, computing a shortest path is required once per episode and typically takes less than one second, which is competitive with existing planning methods. During execution, action selection adds ≈ 1.5 ms overhead per step. We provide a detailed runtime breakdown in Appendix F. In practice, these costs are minor compared to the performance gains, and the absence of additional training allows practitioners to iterate quickly.

The effectiveness of TTGS depends on the base learner. The framework assumes that the policy can reliably reach nearby subgoals, and its performance is influenced by both the accuracy of the distance estimator and the coverage of states in the offline dataset. In some environments we observe substantial improvements from near-zero baselines, but in others the base policies are too unreliable to follow subgoal chains effectively.

We also find that the geometry of the data manifold is critical. In manipulation tasks, where our results are consistent but modest (see Appendix C), evaluation goals often lie outside the main training distribution. As illustrated in Figure 5, this creates a gap in the manifold where no intermediate states exist to support planning. In such sparse or disconnected settings, TTGS naturally defaults to the base policy’s behavior rather than hallucinating invalid paths. Unlike generative planners which may synthesize physically impossible trajectories, TTGS is conservative by design, relying only on retrieved dataset states.

Finally, the effectiveness of TTGS relies on the quality of the underlying distance estimates. When goal-conditioned value functions provide noisy or inconsistent signals, TTGS may struggle to produce useful guidance, though we note that in such regimes the base learners typically perform poorly as well. Furthermore, TTGS may select subgoals that are suboptimal when dataset coverage is limited or dis-

tance predictions are inaccurate. Future work could explore combining multiple noisy distance estimates to obtain more robust edge weights, as well as developing more sophisticated strategies for selecting representative states as graph vertices.

7. Conclusion

We introduced Test-Time Graph Search (TTGS), a lightweight planning framework that augments pretrained goal-conditioned reinforcement learning agents. TTGS leverages only the offline dataset and a distance signal to construct a graph, then applies efficient search to generate subgoal sequences for a frozen policy. This approach requires no retraining, no additional supervision, and no online interaction, yet substantially improves long-horizon performance across diverse tasks.

Our study highlights a key reason why value-based GCRL agents struggle on long-horizon problems: their policies fail to exploit the geometric structure already captured by goal-conditioned value functions. By reinterpreting these value functions as distances and planning over dataset states, TTGS shows that the same policies can achieve significantly stronger performance.

Experiments on OGBench confirm that TTGS consistently enhances the reliability of existing agents, enabling them to solve challenging long-horizon problems that are otherwise difficult with one-shot execution. The method is effective with both value-derived distances and domain-specific metrics, underscoring its flexibility and broad applicability.

By turning offline datasets into a substrate for planning at test time, TTGS provides a practical way to extend the capabilities of goal-conditioned learners without modifying their training pipelines. We hope this perspective inspires further exploration of simple, modular planning strategies that complement learning-based agents and increase their reliability and long-horizon performance.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Abdellatif, A. A., Mhaisen, N., Chkirbene, Z., Mohamed, A., Erbad, A., and Guizani, M. Reinforcement learning for intelligent healthcare systems: A comprehensive survey, 2021. 1
- Ahn, H., Choi, H., Han, J., and Moon, T. Option-aware temporally abstracted value for offline goal-conditioned reinforcement learning. In *NeurIPS*, 2025. 5, 7, 11
- Ajay, A., Du, Y., Gupta, A., Tenenbaum, J. B., Jaakkola, T. S., and Agrawal, P. Is conditional generative modeling all you need for decision making? In *ICLR*, 2023. 1, 8
- Andrychowicz, M., Crow, D., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, P., and Zaremba, W. Hindsight experience replay. In *NeurIPS*, 2017. 1
- Baek, S., Park, T., Park, J., Oh, S., and Kim, Y. Graph-assisted stitching for offline hierarchical reinforcement learning. In *ICML*, 2025. 1, 2, 4, 7, 11, 14
- Chua, K., Calandra, R., McAllister, R., and Levine, S. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *NeurIPS*, 2018. 8
- Emmons, S., Jain, A., Laskin, M., Kurutach, T., Abbeel, P., and Pathak, D. Sparse graphical memory for robust planning. In *NeurIPS*, 2020. 1, 2, 7
- Eysenbach, B., Salakhutdinov, R., and Levine, S. Search on the replay buffer: Bridging planning and reinforcement learning. In *NeurIPS*, 2019. 1, 7
- Eysenbach, B., Zhang, T., Levine, S., and Salakhutdinov, R. Contrastive learning as goal-conditioned reinforcement learning. In *NeurIPS*, 2022. 7
- Fu, J., Kumar, A., Nachum, O., Tucker, G., and Levine, S. D4rl: Datasets for deep data-driven reinforcement learning. *ArXiv preprint*, 2020. 1
- Giammarino, V., Ni, R., and Qureshi, A. H. Physics-informed value learner for offline goal-conditioned reinforcement learning. In *NeurIPS*, 2025. URL <https://openreview.net/forum?id=LRyGqUz7kY>. 7
- Hafner, D., Lillicrap, T. P., Fischer, I., Villegas, R., Ha, D., Lee, H., and Davidson, J. Learning latent dynamics for planning from pixels. In *ICML*, 2019. 8
- Hansen, N., Su, H., and Wang, X. Temporal difference learning for model predictive control. In *ICML*, 2022. 8
- Janner, M., Li, Q., and Levine, S. Offline reinforcement learning as one big sequence modeling problem. In *NeurIPS*, 2021. 8
- Janner, M., Du, Y., Tenenbaum, J. B., and Levine, S. Planning with diffusion for flexible behavior synthesis. In *ICML*, 2022. 8
- Kostrikov, I., Nair, A., and Levine, S. Offline reinforcement learning with implicit q-learning. In *ICLR*, 2022. 2, 7, 11
- Kumar, A., Zhou, A., Tucker, G., and Levine, S. Conservative q-learning for offline reinforcement learning. In *NeurIPS*, 2020. 1
- Lee, K. and Choi, J. State-covering trajectory stitching for diffusion planners. *ArXiv preprint*, 2025. 8
- Luo, Y., Mishra, U. A., Du, Y., and Xu, D. Generative trajectory stitching through diffusion composition. *ArXiv preprint*, 2025. 1, 2, 8, 11, 12
- Park, S., Ghosh, D., Eysenbach, B., and Levine, S. HIQL: offline goal-conditioned RL with latent states as actions. In *NeurIPS*, 2023. 2, 5, 7, 11
- Park, S., Frans, K., Eysenbach, B., and Levine, S. Ogbench: Benchmarking offline goal-conditioned rl. In *ICLR*, 2025. 1, 2, 5
- Savinov, N., Dosovitskiy, A., and Koltun, V. Semi-parametric topological memory for navigation. In *ICLR*, 2018. 7
- Schaul, T., Horgan, D., Gregor, K., and Silver, D. Universal value function approximators. In *ICML, JMLR Workshop and Conference Proceedings*, 2015. 1
- Sobal, V., Zhang, W., Cho, K., Balestriero, R., Rudner, T. G. J., and LeCun, Y. Learning from reward-free offline data: A case for planning with latent dynamics models, 2025. 1
- Wang, T., Torralba, A., Isola, P., and Zhang, A. Optimal goal-reaching reinforcement learning via quasimetric learning. In *ICML*, 2023. 2, 5, 7, 11
- Williams, G., Aldrich, A., and Theodorou, E. A. Model predictive path integral control: From theory to parallel computation. *Journal of Guidance, Control, and Dynamics*, (2), 2017. 8
- Yin, Z.-H. and Abbeel, P. Offline imitation learning through graph search and retrieval. *ArXiv preprint*, 2024. 7
- Zhang, L., Yang, G., and Stadie, B. C. World model as a graph: Learning latent landmarks for planning. In *ICML*, 2021. 7
- Zhang, X., Lin, H., Ye, H., Zou, J., Ma, J., Liang, Y., and Du, Y. Inference-time scaling of diffusion models through classical search. *ArXiv preprint*, 2025. 8

- Zheng, C., Eysenbach, B., Walke, H. R., Yin, P., Fang, K., Salakhutdinov, R., and Levine, S. Stabilizing contrastive RL: techniques for robotic goal reaching from offline data. In *ICLR*, 2024. 7
- Zhou, J. L. and Kao, J. C. Flattening hierarchies with policy bootstrapping. *arXiv preprint arXiv:2505.14975*, 2025. 5, 11
- Zhu, D., Li, L. E., and Elhoseiny, M. Value memory graph: A graph-structured world model for offline reinforcement learning. In *ICLR*, 2023. 7

A. Full Results

Scope. We present results for TTGS paired with HIQL (Park et al., 2023), GCIQL (Kostrikov et al., 2022), QRL (Wang et al., 2023), SAW (Zhou & Kao, 2025), and OTA (Ahn et al., 2025) in Table 3. We compare against each base learner and two planning methods: GAS (Baek et al., 2025) and CompDiffuser (CD) (Luo et al., 2025). This comparison is not apples-to-apples: our TTGS results use only the offline pretraining dataset and frozen base agent components, while GAS and CD include additional learned modules and training. We use value-derived distances for TTGS in this comparison. Results are summarized in Table 3.

Setting differences. TTGS operates in a *more constrained* setting than both GAS and CompDiffuser: we perform no additional training and use only the offline pretraining dataset along with the pretrained goal-conditioned value function and frozen policy of each base learner. By contrast, GAS trains a Temporal Distance Representation (TDR) and leverages it to train a low-level controller with subgoal supervision, while CompDiffuser trains a diffusion model to synthesize long-horizon trajectories. These differences mean TTGS adds purely test-time computation, whereas GAS and CD introduce extra learned components, while also requiring planning at test time.

Table 3. Success rates (%) across datasets. We report base learners, their TTGS-augmented counterparts, and other planning agents where available. “/” indicates the metric was not reported in the corresponding paper. Means and standard deviations follow the OGBench protocol (50 rollouts per task; averages and s.d. over 8 seeds).

Dataset	HIQL	HIQL +TTGS	GCIQL	GCIQL +TTGS	QRL	QRL +TTGS	SAW	SAW +TTGS	OTA	OTA +TTGS	GAS	CD
pointmaze-medium-navigate-v0	73.6 ± 4.4	<u>85.8 ± 5.2</u>	51.5 ± 8.2	<u>90.9 ± 3.7</u>	83.5 ± 3.2	98.4 ± 3.4	96.8 ± 1.8	96.8 ± 1.9	80.8 ± 3.1	<u>92.4 ± 3.6</u>	/	/
pointmaze-medium-stitch-v0	73.0 ± 10.2	<u>80.5 ± 20.3</u>	18.2 ± 8.9	<u>44.0 ± 8.0</u>	76.0 ± 8.9	<u>93.4 ± 6.8</u>	68.2 ± 7.6	<u>85.4 ± 6.1</u>	60.6 ± 11.2	<u>81.0 ± 8.7</u>	/	100 ± 0
pointmaze-large-navigate-v0	45.2 ± 14.0	<u>78.0 ± 9.7</u>	32.2 ± 5.8	88.1 ± 9.4	82.4 ± 5.7	92.0 ± 9.7	78.0 ± 10.8	<u>83.6 ± 6.4</u>	85.1 ± 5.7	87.7 ± 6.6	/	/
pointmaze-large-stitch-v0	13.2 ± 8.0	<u>92.2 ± 5.1</u>	29.0 ± 5.0	29.0 ± 2.2	88.8 ± 14.0	<u>93.9 ± 8.7</u>	41.2 ± 6.7	<u>93.2 ± 6.1</u>	64.0 ± 14.3	<u>69.8 ± 18.5</u>	/	100 ± 0
pointmaze-giant-navigate-v0	43.0 ± 10.5	<u>70.9 ± 12.2</u>	0.0 ± 0.0	91.9 ± 3.0	65.3 ± 10.2	<u>88.1 ± 9.5</u>	68.5 ± 6.4	94.8 ± 2.8	67.2 ± 9.8	<u>73.6 ± 7.1</u>	/	/
pointmaze-giant-stitch-v0	0.0 ± 0.0	<u>80.9 ± 9.0</u>	0.0 ± 0.0	98.0 ± 1.4	55.3 ± 12.0	93.2 ± 7.7	6.8 ± 7.9	<u>82.3 ± 13.6</u>	42.9 ± 9.6	<u>86.8 ± 5.5</u>	/	68 ± 3
antmaze-medium-navigate-v0	95.2 ± 0.7	95.2 ± 1.3	72.3 ± 4.7	<u>81.1 ± 4.7</u>	81.9 ± 10.6	<u>83.9 ± 7.8</u>	96.3 ± 1.6	95.7 ± 1.7	95.6 ± 1.5	95.6 ± 1.5	96.3 ± 1.3	/
antmaze-medium-stitch-v0	92.9 ± 2.1	95.4 ± 1.3	29.5 ± 4.7	<u>53.0 ± 8.7</u>	62.0 ± 9.4	40.1 ± 9.4	64.0 ± 5.0	94.0 ± 1.9	88.5 ± 2.7	<u>91.1 ± 3.2</u>	98.1 ± 1.2	96 ± 2
antmaze-large-navigate-v0	90.6 ± 2.5	92.3 ± 2.3	35.8 ± 2.7	<u>57.2 ± 3.8</u>	74.0 ± 4.3	<u>77.0 ± 3.3</u>	88.8 ± 2.5	89.6 ± 1.7	91.4 ± 1.0	91.6 ± 1.4	93.2 ± 0.5	/
antmaze-large-stitch-v0	73.0 ± 6.0	<u>90.8 ± 2.3</u>	7.0 ± 2.5	<u>30.6 ± 4.6</u>	21.0 ± 4.1	<u>43.5 ± 10.5</u>	3.1 ± 4.8	<u>86.2 ± 3.7</u>	85.4 ± 3.8	<u>91.4 ± 1.5</u>	96.3 ± 0.9	86 ± 2
antmaze-giant-navigate-v0	65.0 ± 4.1	<u>65.8 ± 4.0</u>	0.4 ± 0.2	<u>5.4 ± 2.4</u>	11.8 ± 5.2	10.8 ± 4.8	68.5 ± 3.0	<u>71.0 ± 5.3</u>	67.4 ± 6.1	<u>68.5 ± 4.1</u>	76.0 ± 5.9	/
antmaze-giant-stitch-v0	1.4 ± 1.1	<u>78.6 ± 13.4</u>	0.0 ± 0.0	<u>32.7 ± 6.6</u>	2.0 ± 2.6	<u>52.2 ± 23.6</u>	0.0 ± 0.0	<u>36.8 ± 19.5</u>	35.2 ± 6.1	<u>71.0 ± 4.6</u>	86.2 ± 3.6	65 ± 3
antmaze-large-explore-v0	2.4 ± 4.4	<u>26.6 ± 34.0</u>	0.2 ± 0.5	<u>66.7 ± 8.3</u>	0.0 ± 0.1	0.0 ± 0.0	1.9 ± 1.8	91.8 ± 4.6	80.8 ± 5.9	80.8 ± 13.9	91.0 ± 9.4	27 ± 1
humanoidmaze-medium-navigate-v0	88.5 ± 3.0	93.3 ± 2.9	31.8 ± 3.8	<u>50.5 ± 5.5</u>	19.3 ± 8.1	<u>19.7 ± 8.2</u>	87.9 ± 2.9	86.9 ± 3.1	80.8 ± 5.9	80.8 ± 13.9	96.3 ± 0.4	/
humanoidmaze-medium-stitch-v0	86.1 ± 3.0	84.6 ± 2.5	14.0 ± 3.6	<u>20.2 ± 6.8</u>	19.4 ± 5.0	14.4 ± 3.0	63.6 ± 2.2	95.0 ± 0.9	88.4 ± 3.3	83.9 ± 4.9	96.2 ± 1.6	91 ± 1
humanoidmaze-large-navigate-v0	48.0 ± 4.7	<u>79.4 ± 5.6</u>	2.1 ± 1.1	<u>8.4 ± 3.6</u>	6.8 ± 2.0	<u>10.0 ± 2.7</u>	46.8 ± 6.8	<u>64.3 ± 6.0</u>	78.8 ± 4.3	86.4 ± 3.5	84.6 ± 3.7	/
humanoidmaze-large-stitch-v0	28.6 ± 2.9	<u>65.1 ± 10.7</u>	0.7 ± 0.5	<u>2.0 ± 0.7</u>	3.9 ± 2.5	3.1 ± 1.5	11.6 ± 5.3	<u>75.6 ± 12.1</u>	58.2 ± 4.9	<u>68.9 ± 9.2</u>	80.6 ± 2.3	72 ± 3
humanoidmaze-giant-navigate-v0	16.0 ± 8.6	<u>85.3 ± 6.1</u>	0.7 ± 0.3	0.5 ± 2.0	1.1 ± 0.5	1.0 ± 1.0	40.4 ± 2.7	<u>79.0 ± 5.3</u>	89.1 ± 3.9	92.8 ± 2.1	85.7 ± 2.5	/
humanoidmaze-giant-stitch-v0	4.4 ± 1.3	<u>78.1 ± 5.1</u>	0.2 ± 0.3	0.2 ± 0.5	0.5 ± 0.4	<u>4.1 ± 2.4</u>	0.0 ± 0.1	<u>79.8 ± 5.6</u>	81.6 ± 2.8	88.8 ± 2.6	82.4 ± 2.5	67 ± 4
visual-antmaze-large-navigate-v0	72.0 ± 3.1	83.8 ± 2.6	2.4 ± 0.6	<u>3.6 ± 0.6</u>	0.9 ± 2.4	0.3 ± 0.7	72.2 ± 4.6	<u>75.8 ± 5.2</u>	60.2 ± 7.9	<u>67.1 ± 12.2</u>	85.2 ± 6.6	/
visual-antmaze-giant-navigate-v0	5.2 ± 5.2	<u>7.6 ± 7.7</u>	0.5 ± 0.4	0.2 ± 0.3	0.1 ± 0.3	<u>3.3 ± 3.2</u>	4.2 ± 1.2	<u>8.4 ± 4.0</u>	10.0 ± 6.8	<u>19.0 ± 12.1</u>	67.2 ± 3.0	/
visual-antmaze-large-stitch-v0	28.7 ± 5.5	<u>66.4 ± 16.0</u>	0.1 ± 0.3	0.0 ± 0.0	0.4 ± 0.5	0.4 ± 0.6	6.6 ± 7.6	<u>60.4 ± 36.4</u>	22.1 ± 5.2	<u>64.6 ± 9.3</u>	77.2 ± 6.1	/
visual-antmaze-giant-stitch-v0	0.2 ± 0.4	<u>32.2 ± 17.3</u>	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.1	<u>6.0 ± 3.1</u>	0.0 ± 0.0	<u>31.0 ± 24.9</u>	2.2 ± 1.9	<u>35.2 ± 9.1</u>	51.9 ± 6.4	/
visual-antmaze-medium-explore-v0	0.2 ± 0.6	<u>63.0 ± 28.1</u>	0.0 ± 0.0	0.0 ± 0.0	0.7 ± 1.0	<u>0.8 ± 1.5</u>	2.4 ± 3.4	<u>58.0 ± 26.7</u>	30.0 ± 15.5	80.5 ± 34.0	65.9 ± 6.8	/
visual-antmaze-large-explore-v0	0.0 ± 0.0	<u>0.8 ± 1.6</u>	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	<u>15.2 ± 13.5</u>	3.6 ± 3.9	25.2 ± 13.7	15.1 ± 6.8	/
visual-humanoidmaze-medium-navigate-v0	0.8 ± 0.9	0.4 ± 0.5	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.1 ± 0.2	<u>0.2 ± 0.2</u>	1.5 ± 1.4	0.9 ± 0.9	/	/
visual-humanoidmaze-medium-stitch-v0	0.4 ± 0.5	<u>0.7 ± 0.6</u>	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.2 ± 0.3	0.2 ± 0.4	0.8 ± 0.4	0.8 ± 0.7	/	/

Findings. Despite requiring no additional training, TTGS improves the performance of the base learners in the vast majority of cases. On several hardest tasks (humanoidmaze-giant-navigate-v0, humanoidmaze-giant-stitch-v0, visual-antmaze-large-explore-v0) OTA+TTGS outperforms more complex planning baselines that rely on extra training. These gains support our central claim: simple metric-guided test-time planning can unlock long-horizon competence already latent in value-based GCRL agents.

Reproduction details and caveats. The environment provided by OGBench has random start and goal positions even when the task ID and seed are fixed. This explains some variance in the results across tables. CompDiffuser results are taken directly from Luo et al. (2025). GAS results were taken from the official implementation repository <https://github.com/qortmdgh4141/GAS>, since humanoidmaze results were not reported in the paper.

Comparison with GAS using fixed hyperparameters. To compare the sensitivity to hyperparameters with GAS, we compare HIQL+TTGS using a single fixed set of hyperparameters tuned for antmaze across other domains. TTGS uses ($\tau = 24, T = 48$), GAS uses official implementation with antmaze hyperparameters, both evaluated over 8 random seeds. Results are reported in Table 4. While GAS achieves higher scores on tuned environment, TTGS outperforms GAS on humanoidmaze and pointmaze, highlighting the robustness of TTGS across diverse domains despite using fixed hyperparameters, which might be important in a fully offline setting or when tuning is expensive.

Table 4. Success rates on OGBench locomotion tasks for HIQL+TTGS and GAS using a single fixed set of hyperparameters derived from antmaze. While GAS achieves higher scores on antmaze (the domain it is tuned for), TTGS with fixed hyperparameters drastically outperforms GAS on humanoidmaze and pointmaze, highlighting TTGS as a more robust cross-domain planner.

Dataset	HIQL	HIQL+TTGS (Fixed)	GAS (Fixed)
pointmaze-giant-navigate-v0	47 $\pm$ 10	68 $\pm$ 13	5 $\pm$ 11
pointmaze-giant-stitch-v0	0 $\pm$ 0	73 $\pm$ 14	0 $\pm$ 0
antmaze-giant-navigate-v0	67 $\pm$ 4	68 $\pm$ 3	76 $\pm$ 6
antmaze-giant-stitch-v0	2 $\pm$ 1	48 $\pm$ 19	86 $\pm$ 4
antmaze-large-explore-v0	5 $\pm$ 7	60 $\pm$ 26	91 $\pm$ 9
humanoidmaze-giant-navigate-v0	14 $\pm$ 6	78 $\pm$ 9	14 $\pm$ 5
humanoidmaze-giant-stitch-v0	4 $\pm$ 2	69 $\pm$ 18	8 $\pm$ 4

B. Value function to distance mappings

Value-to-distance transformation for HIQL, SAW, and GCIQL. HIQL, SAW, and GCIQL use a per-step penalty, that is reward -1 until the goal and 0 at the goal. As discussed in Section 3.1, we map goal-conditioned values to predicted step counts via

$$d(s, g) = \log_\gamma(1 + (1 - \gamma) V_{\text{clip}}(s, g)),$$

where $\gamma \in (0, 1)$ is the discount factor. To avoid infinite outputs, we clip value predictions to the open interval $(-\frac{1}{1-\gamma}, 0)$ with a small margin $\varepsilon = 10^{-3}$:

$$V_{\text{clip}}(s, g) = \min(-\varepsilon, \max(V(s, g), -\frac{1}{1-\gamma} + \varepsilon)).$$

We then apply the transform to V_{clip} . For HIQL and SAW we average the two value heads, $V(s, g) = \frac{1}{2}(V_1(s, g) + V_2(s, g))$. For GCIQL we use its single value head $V(s, g)$.

OTA distance. OTA learns two goal-conditioned value functions at different temporal resolutions: a low-level critic $V^{\text{low}}(s, g)$ defined over primitive environment steps, and a high-level critic $V^{\text{high}}(s, g)$ defined over temporally abstracted option transitions of length n environment steps. Both critics use two value heads; We average them and apply the same clipping to the open interval $(-\frac{1}{1-\gamma}, 0)$ with margin ε as above, then inverting the geometric-sum return for the negative reward semantics via

$$\hat{d}(V; \gamma) = \log_\gamma(1 + (1 - \gamma) V).$$

For the low-level critic this directly yields an environment-step estimate $d_{\text{low}}(s, g) = \hat{d}(V^{\text{low}}(s, g); \gamma_{\text{low}})$. For the high-level critic, the value is defined at option boundaries and corresponds to accumulating a penalty -1 for each *unsuccessful* option and 0 for the final successful option; therefore $\hat{d}$ recovers the expected number of failed options $k_{\text{opt}}(s, g) = \hat{d}(V^{\text{high}}(s, g); \gamma_{\text{high}})$. This means that $k_{\text{opt}}(s, g) + 1$ is expected amount of options to reach the goal. It can be converted to environment steps by

$$d_{\text{high}}(s, g) = n(k_{\text{opt}}(s, g) + 1).$$

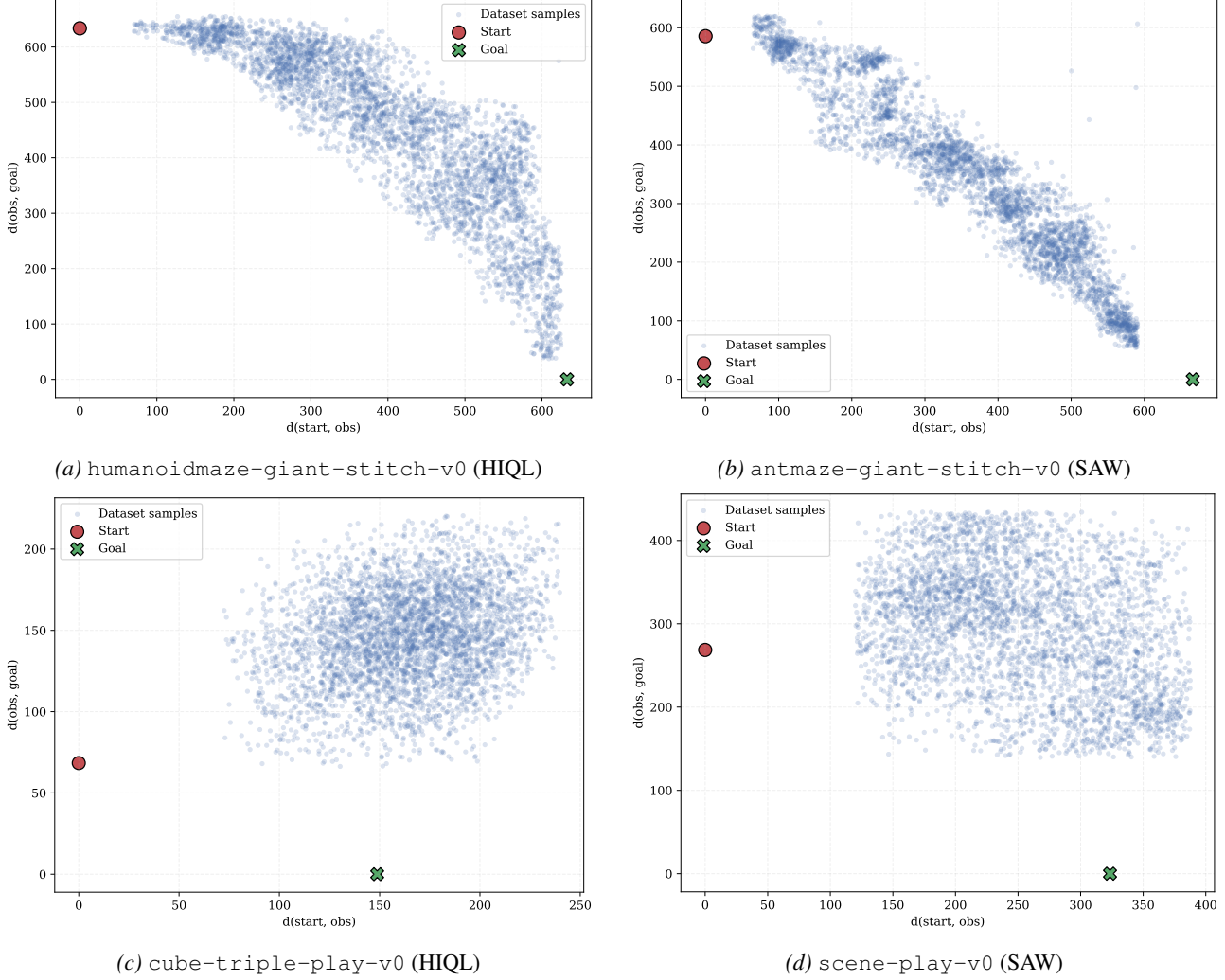


Figure 5. Value Function Geometry. In locomotion tasks (a, b), dataset states form a connected band between start and goal (low $x + y$), supporting planning. In manipulation tasks (c, d), evaluation goals often lie far from the data distribution, creating a gap where no intermediate states exist. TTGS correctly identifies these high-cost edges and defaults to the base policy rather than hallucinating paths.

Finally, we combine both distances into a single environment-step prediction by using d_{low} for short horizons and gradually switching to the more stable d_{high} for long horizons, matching the training distributions of both value functions. We use d_{low} when it predicts distances below s_0 (subgoal_steps) used to train the actor policies of OTA and linearly interpolate to d_{high} with n as the ramp width, where n is the abstraction_factor hyperparameter of OTA:

$$w = \text{clip}\left(\frac{d_{\text{low}}(s, g) - s_0}{n}, 0, 1\right), \quad d(s, g) = (1 - w) d_{\text{low}}(s, g) + w \max(d_{\text{high}}(s, g), d_{\text{low}}(s, g)).$$

QRL distance. Quasi-metric RL learns a nonnegative function $d_{\text{qrl}}(s, g)$ that approximates the number of steps from s to g . This head already represents a distance, so we use it directly without additional transformation.

Minimum step constraint. In all cases we lower bound predicted distances by 1 step, since no transition can be completed with fewer than one action.

C. Manipulation Tasks and Value Function Geometry

We evaluate TTGS on OGBench manipulation tasks in Table 5, pairing it with the strongest base agent provided by OGBench for these domains, GCIQL. We observe that improvements are consistent but modest compared to locomotion tasks.

Table 5. **Success rates on OGBench manipulation tasks.** Improvements are modest because manipulation datasets lack intermediate states connecting start and evaluation goals, so TTGS often correctly defaults to the base policy instead of hallucinating unsupported plans.

Dataset	GCIQL Success	GCIQL+TTGS Success
scene-play-v0	50 ± 7	52 ± 4
cube-triple-play-v0	4 ± 2	4 ± 2
puzzle-4x5-play-v0	12 ± 3	13 ± 3
puzzle-4x6-play-v0	10 ± 0	10 ± 0

To understand this behavior, we analyze the geometry of the learned value functions. In Figure 5, we visualize the distribution of the sampled dataset states on a 2D plane defined by the predicted distance from the start state (x -axis) and the predicted distance to the goal (y -axis).

In locomotion tasks (humanoidmaze, antmaze), states typically form a curve connecting the start and goal (low $x + y$), providing a dense set of intermediate subgoals for the graph search. In contrast, for manipulation tasks (cube-triple, scene), the dataset states often cluster far from both the start and the evaluation goal, with very few states bridging the gap. This indicates that the evaluation goals lie outside the main data manifold. Crucially, TTGS correctly identifies that no reliable path exists through the graph—due to high edge costs across the manifold gap—and defaults to the base policy behavior. This confirms that TTGS acts as a safe planning wrapper that does not hallucinate plans when data support is missing.

D. Alternative Training Data Filtering

We tested a vertex sampling scheme based on clustering to improve state coverage, inspired by Baek et al. (2025). While Baek et al. (2025) filter states by temporal efficiency before clustering, we found that adapting this metric to standard value functions was ineffective in preliminary runs. Consequently, we compare standard TTGS (which uses uniform random sampling for graph vertices) against a variant that selects vertices by clustering a large pool of randomly sampled states.

Clustering. We sample a large subset $\mathcal{H}$ of 80,000 states uniformly from $\mathcal{D}$. We select graph vertices $\mathcal{V} \subset \mathcal{H}$ using a single-pass greedy clustering rule with radius $r = \tau/2$:

1. Initialize $\mathcal{V} \leftarrow \{s^{(0)}\}$, the first element of $\mathcal{H}$.
2. For each $x \in \mathcal{H}$, compute $m = \min_{v \in \mathcal{V}} \hat{d}(x, v)$.
3. If $m > r$, add x to $\mathcal{V}$. Otherwise assign x to its nearest center and periodically update that center to the member that minimizes total within-cluster distance.

Table 6 compares the performance and runtime of TTGS with random sampling versus clustering. We observe no statistically significant performance gain from clustering, while the computational cost of graph construction increases by an order of magnitude (from ~ 40 seconds to > 10 minutes). Thus, we adopt random sampling as the default for its efficiency.

Table 6. Effect of state clustering on TTGS. We compare TTGS with and without state clustering. Clustering uses a random subset of 80,000 states, clustering distance threshold $\frac{1}{2}\tau$, and a maximum of 4,000 cluster centers. From the empirical results, clustering provides only marginal gains while its computational overhead is dominant in the overall cost.

Environment	HIQL	HIQL+TTGS w/o Clustering	HIQL+TTGS w/ Clustering	Clustering Time (s)	Cluster Centers
pointmaze-giant-navigate-v0	43 $\pm$ 12	73 $\pm$ 12	74 $\pm$ 9	814.76	613.0
pointmaze-giant-stitch-v0	0 $\pm$ 0	80 $\pm$ 6	79 $\pm$ 10	876.43	3904.88
antmaze-giant-navigate-v0	64 $\pm$ 4	67 $\pm$ 3	69 $\pm$ 3	706.77	4000.0
antmaze-giant-stitch-v0	2 $\pm$ 1	70 $\pm$ 14	69 $\pm$ 11	1932.88	4000.0
antmaze-large-explore-v0	2 $\pm$ 4	26 $\pm$ 34	25 $\pm$ 33	771.11	4000.0
humanoidmaze-giant-navigate-v0	15 $\pm$ 7	85 $\pm$ 6	83 $\pm$ 9	691.54	4000.0
humanoidmaze-giant-stitch-v0	4 $\pm$ 2	78 $\pm$ 8	78 $\pm$ 12	705.48	4000.0

E. Soft vs. Hard Edge Penalties

We investigate whether the soft penalty scheme is necessary compared to a simpler hard thresholding scheme. We compare our method (Soft Penalty) against a variant where edges with predicted distance greater than τ are removed entirely (Hard Threshold).

Table 7 presents the results. On simple navigation tasks, the Hard Threshold often performs comparably to the Soft Penalty. However, on tasks requiring trajectory stitching, the Hard Threshold leads to drastic performance drops. For example, success drops from 78.6% to 2.2% on antmaze-giant-stitch-v0 and from 78.1% to 14.0% on humanoidmaze-giant-stitch-v0.

This performance gap is explained by the *Disconnect Ratio*, defined as the fraction of episodes where the goal becomes unreachable from the start state in the constructed graph. Standard value functions are noisy and may underestimate the cost of specific transitions required to bridge disparate trajectories in the dataset. A hard threshold disconnects these essential links, rendering planning impossible. The soft penalty allows these edges to persist with high cost, preserving connectivity while still discouraging their use unless no safer path exists.

Table 7. Comparison of Soft vs. Hard Penalties. The Hard Threshold variant frequently disconnects the graph on stitching tasks (high Disconnect Ratio), leading to poor success rates. The Soft Penalty maintains connectivity by assigning high costs to optimistic edges rather than removing them. Results are for HIQL+TTGS.

Dataset	HIQL+TTGS (Soft)	HIQL+TTGS (Hard)	Disconnect Ratio
pointmaze-giant-navigate-v0	71 $\pm$ 12	65 $\pm$ 10	0.08
pointmaze-giant-stitch-v0	81 $\pm$ 9	66 $\pm$ 9	0.25
antmaze-giant-navigate-v0	66 $\pm$ 4	67 $\pm$ 4	0.11
antmaze-giant-stitch-v0	79 $\pm$ 13	2 $\pm$ 2	0.99
antmaze-large-explore-v0	27 $\pm$ 34	53 $\pm$ 32	0.00
humanoidmaze-giant-navigate-v0	85 $\pm$ 6	57 $\pm$ 9	0.47
humanoidmaze-giant-stitch-v0	78 $\pm$ 5	14 $\pm$ 14	0.84
visual-antmaze-large-navigate-v0	84 $\pm$ 3	64 $\pm$ 9	0.38
visual-antmaze-large-stitch-v0	66 $\pm$ 16	31 $\pm$ 14	0.54

F. Runtime Analysis

We provide a detailed breakdown of the computational overhead of TTGS in Table 8. All runtimes were measured on a single Nvidia L40s GPU with 8 virtual CPU cores. Graph construction is performed once per dataset, and the shortest path search is performed once per episode (or sparingly if replanning). Both operations are matrix-heavy and highly parallelizable; we implement them on the GPU. The per-episode planning overhead is generally around 1 second or less. The per-step action selection overhead is negligible (≈ 1.5 ms).

Table 8. TTGS Runtime Overhead. Per-environment cost of graph construction and online planning components in TTGS. Measured on an Nvidia L40s GPU with 8 vCPUs using HIQL as base agent. Graph build is one-time compute for all the tasks, while shortest path is one-time compute per episode.

Environment	Graph Construction (s)	Shortest Path (s)	Pick Subgoal (ms)	Step Overhead (ms)
pointmaze-giant-navigate-v0	36.42	1.25	0.36	1.68
pointmaze-giant-stitch-v0	34.98	0.79	0.37	1.61
antmaze-giant-navigate-v0	36.55	0.46	0.37	1.42
antmaze-giant-stitch-v0	36.26	0.84	0.39	1.66
antmaze-large-explore-v0	36.40	0.46	0.38	1.33
humanoidmaze-giant-navigate-v0	34.99	0.59	0.38	1.39
humanoidmaze-giant-stitch-v0	36.35	1.0	0.39	1.38
visual-antmaze-large-navigate-v0	100.02	0.53	0.47	2.63
visual-antmaze-large-stitch-v0	100.89	0.53	0.48	2.49

G. Replanning Analysis

We evaluate a variant of TTGS that triggers a full recalculation of the shortest path if the agent deviates from the current subgoal by more than $2T$. To prevent excessive path oscillations, we limit replanning to occur at most once every 50 steps. Table 9 compares this against the standard one-time planning approach. Results show that while replanning can help in datasets with highly noisy trajectories (e.g., `antmaze-large-explore`), the adaptive subgoal selection on the fixed guide path is sufficient and often superior for most tasks.

Table 9. Effect of replanning in TTGS. Comparison between the default TTGS (one-time planning) with a variant that replans when the agent strays $> 2T$ from the current subgoal (at most once every 50 steps). *Replan Episode Ratio* indicates the fraction of episodes where replanning occurred. In most environments, the one-time adaptive guide is sufficient, and frequent replanning can degrade performance by causing oscillations.

Environment	HIQL	HIQL+TTGS w/o Replan	HIQL+TTGS w/ Replan	Replan Episode Ratio
pointmaze-giant-navigate-v0	50 ± 13	73 ± 12	72 ± 6	0.01
pointmaze-giant-stitch-v0	0 ± 0	80 ± 6	80 ± 6	0.00
antmaze-giant-navigate-v0	66 ± 3	67 ± 3	71 ± 2	0.52
antmaze-giant-stitch-v0	1 ± 1	70 ± 14	67 ± 13	0.38
antmaze-large-explore-v0	2 ± 3	26 ± 34	50 ± 32	0.07
humanoidmaze-giant-navigate-v0	14 ± 5	85 ± 6	78 ± 4	0.52
humanoidmaze-giant-stitch-v0	4 ± 3	78 ± 8	33 ± 10	0.98
visual-antmaze-large-navigate-v0	46 ± 14	82 ± 7	70 ± 10	0.86
visual-antmaze-large-stitch-v0	18 ± 8	78 ± 4	53 ± 24	0.76

H. Hyperparameters

TTGS does not require training and has only three hyperparameters, so tuning is quick and simple. We did not run extensive sweeps; Table 10 lists the values used in our experiments.

M is the number of dataset states sampled as graph vertices. Larger M increases coverage and cost. In our preliminary experiments we found $M = 4000$ to be sufficient while performant, with M as low as 100 helpful for improving base agent’s performance.

Table 10. Hyperparameter settings used in our experiments.

Dataset	M	HIQL (τ, T)	QRL (τ, T)	GCIQL (τ, T)	SAW (τ, T)	OTA (τ, T)
pointmaze-medium-navigate-v0	4000	(24, 48)	(24, 48)	(24, 48)	(24, 48)	(24, 48)
pointmaze-medium-stitch-v0	4000	(24, 48)	(24, 48)	(24, 48)	(24, 48)	(24, 48)
pointmaze-large-navigate-v0	4000	(24, 48)	(24, 48)	(24, 48)	(24, 48)	(24, 48)
pointmaze-large-stitch-v0	4000	(24, 48)	(24, 48)	(48, 96)	(24, 48)	(24, 48)
pointmaze-giant-navigate-v0	4000	(24, 48)	(24, 48)	(12, 24)	(24, 48)	(24, 48)
pointmaze-giant-stitch-v0	4000	(12, 24)	(12, 24)	(6, 12)	(24, 48)	(24, 48)
antmaze-medium-navigate-v0	4000	(24, 48)	(24, 48)	(24, 48)	(24, 48)	(24, 48)
antmaze-medium-stitch-v0	4000	(24, 48)	(24, 48)	(24, 48)	(24, 48)	(24, 48)
antmaze-large-navigate-v0	4000	(24, 48)	(24, 48)	(24, 48)	(24, 48)	(24, 48)
antmaze-large-stitch-v0	4000	(24, 48)	(24, 48)	(24, 48)	(24, 48)	(24, 48)
antmaze-giant-navigate-v0	4000	(24, 48)	(12, 24)	(12, 24)	(24, 48)	(24, 48)
antmaze-giant-stitch-v0	4000	(12, 24)	(12, 24)	(12, 24)	(24, 48)	(24, 48)
antmaze-large-explore-v0	4000	(24, 48)	(24, 48)	(24, 48)	(24, 48)	(24, 48)
humanoidmaze-medium-navigate-v0	4000	(24, 48)	(24, 48)	(24, 48)	(24, 48)	(48, 96)
humanoidmaze-medium-stitch-v0	4000	(24, 48)	(24, 48)	(24, 48)	(24, 48)	(48, 96)
humanoidmaze-large-navigate-v0	4000	(24, 48)	(24, 48)	(24, 48)	(24, 48)	(48, 96)
humanoidmaze-large-stitch-v0	4000	(24, 48)	(24, 48)	(24, 48)	(24, 48)	(48, 96)
humanoidmaze-giant-navigate-v0	4000	(36, 72)	(24, 48)	(24, 48)	(24, 48)	(48, 96)
humanoidmaze-giant-stitch-v0	4000	(24, 48)	(24, 48)	(12, 24)	(24, 48)	(48, 96)
visual-antmaze-large-navigate-v0	4000	(12, 24)	(12, 24)	(12, 24)	(24, 48)	(24, 48)
visual-antmaze-giant-navigate-v0	4000	(12, 24)	(12, 24)	(12, 24)	(24, 48)	(24, 48)
visual-antmaze-large-stitch-v0	4000	(12, 24)	(12, 24)	(12, 24)	(24, 48)	(24, 48)
visual-antmaze-giant-stitch-v0	4000	(12, 24)	(12, 24)	(12, 24)	(24, 48)	(24, 48)
visual-antmaze-medium-explore-v0	4000	(12, 24)	(12, 24)	(12, 24)	(24, 48)	(24, 48)
visual-antmaze-large-explore-v0	4000	(12, 24)	(12, 24)	(12, 24)	(24, 48)	(24, 48)
visual-humanoidmaze-medium-navigate-v0	4000	(12, 24)	(12, 24)	(12, 24)	(24, 48)	(48, 96)
visual-humanoidmaze-medium-stitch-v0	4000	(12, 24)	(12, 24)	(12, 24)	(24, 48)	(48, 96)

τ is the edge-length threshold used to compute graph weights: distances greater than τ are penalized. A larger τ permits longer hops and is suitable only when long-range distance estimates are accurate and the pretrained policy can reliably traverse them.

T is the maximum allowed distance between the current state and the selected subgoal at execution time. Choose T near the range where the frozen policy is most reliable.

Hyperparameter selection methodology. We selected (τ, T) using a lightweight manual sweep rather than an extensive grid search. In practice we evaluated a small set of candidate pairs following a doubling schedule, e.g., $(\tau, T) \in \{(12, 24), (24, 48), (48, 96)\}$, which keeps the search space compact while covering substantially different hop lengths. Usually, we tried 1 or 2 hyperparameter settings per dataset and base agent. For each candidate (τ, T) , we first constructed the TTGS graph and qualitatively checked the resulting shortest paths: (i) whether paths contain “blank” segments indicative of poor connectivity or unreliable long edges, and (ii) whether paths become excessively fragmented into too many short hops (suggesting an overly conservative threshold). We then ran 1 random seed with promising parameters and selected the pair that produced stable, interpretable paths and the strongest improvement in task performance. We didn’t tune SAW across environments and still observe reasonable performance. Since the sweep involves only a handful of settings, we encourage users to repeat this procedure for new environments or base agents, as the optimal trade-off between long hops and reliability can vary.

I. Use of Generative AI

LLMs were used to revise and polish writing on a single-paragraph scale.